

```

rot_new    = d_rot_new:d_rot_new:Nrot;    % counterpart of "t = dt:dt:T";

d_ord      = 1/Nrot;                      % order resolution, also Nsamp/Nrot
ord_new    = 0:d_ord:Nsamp_new/2;        % order axis

figure;
subplot(211); plot(rot_new,x_env); grid on;
                xlabel('Angle [Rotations]'); ylabel('Amplitude');
title('Hilbert FD filtering: Envelope of Resampled Signal (user ZOOM)');
                set(gca,'fontsize',8);

subplot(212); plot(ord_new,X); grid on;
                xlabel('Order'); ylabel('Amplitude');
title('Hilbert FD filtering: Envelope Order Spectrum (user ZOOM)');
                set(gca,'fontsize',8);

```

4.3.1.10 Summary

Concluding, calculation of **Envelope Spectrum** (ES) from resampled signal, resulting in a so-called “**Envelope Order Spectrum**” (EOS) enables automatic tracking of clear, speed-independent characteristic orders. However, comparing a pair of Figure 4.40 and Figure 4.41 (frequency-domain calculations) with corresponding pair of Figure 4.42 and Figure 4.43 (order-domain calculations) this benefit is at the cost of some loss of envelope characteristic patterns in angle-domain vs. time-domain, which take place during transformation from time-domain to angle-domain.

4.3.2 Velocity and displacement spectrum from acceleration signal

4.3.2.1 Overview

Currently, most of vibration signals are recorded as acceleration due to versatile features of accelerometers, both, classical piezoelectric and modern MEMS designs. In some cases, however, it is recommended to use velocity signal, which needs to be generated synthetically from acceleration digital data. Velocity signals are especially useful to analyze relatively low-frequency components, like shaft components. Worth mentioning, velocity spectrum is generally a poor choice for analysis of structural resonances, like REB diagnostics or cavitation diagnostics.

Current Section illustrates calculation of velocity spectrum in frequency domain and in order domain, i.e. from resampled signal. The calculations are illustrated with the use of simple simulated signals followed by real signal from rotary machinery working in non-stationary conditions. Additionally, the Section covers so-called “omega arithmetics” as a quick way of velocity spectrum calculation directly in the frequency domain.

4.3.2.2 Algorithms in a nutshell

A complete calculation and presentation of a velocity spectrum in the frequency domain involves following steps:

- 1) HP filtration (optional a priori DC component removal)
- 2) Integration
- 3) Scaling (acceleration-to-velocity)
- 4) Calculation of spectral amplitudes (Hz)
- 5) Calculation of spectral resolution (Hz)
- 6) Velocity spectrum display

Generally, step 1 and 2 might be realized with use of different parameters or even with different techniques.

In order domain, the algorithm is slightly extended, and includes following steps:

- 1) HP filtration (optional a priori DC component removal)
- 2) Integration
- 3) Scaling (acceleration-to-velocity)
- 4) **Resampling**
- 5) Calculation of spectral amplitudes (order domain)
- 6) Calculation of spectral resolution (order domain)
- 7) Velocity spectrum display (order domain)

Individual operations, which could be treated independently, are described below. Note that in some cases, they refer to signal processing techniques introduced earlier in this Chapter; therefore, corresponding references are given.

4.3.2.3 *Signal processing blocks*

4.3.2.3.1 HP filtration

Filtering functions are described in details in Section 4.2.1. Modern vibration-based CMS frequently take advantage of the DSP libraries, and use frequency domain filtering. Within the current Chapter, the function `Filter_signal_HP` from Section 4.2.1.7 is used:

```
function [x] = Filter_signal_HP(x,fs,cutoff)
% INPUT
% x      - vector of real time signal <1xN>
% fs     - sampling frequency
% cutoff - High pass filter cutoff frequency

% OUTPUT
% x      - vector of integrated real time signal <1xN>
```

Naturally, in each case, frequency-domain filtering could be replaced by time-domain filtering using various techniques described in Section 4.2.1. The comparison of frequency-domain filtering and time-domain filtering within velocity spectrum generation is demonstrated in Section 4.3.2.

4.3.2.3.2 Integration

As mentioned in Section 4.2.3, integration could be performed in many formalized ways [1]. However, for diagnostic purposes, a simple method could be used, because individual integration technique has typically less influence on the final “shape” of Velocity Spectrum than other parts of the algorithm. This simplified integration is also given in Section 4.2.3:

```
function [xInt] = Calculate_Integrated(x,fs)
% INPUT
% x      - vector of real time signal <1xN>
% OUTPUT
% x      - vector of integrated real time signal <1xN>
```

A comparison of this rectangular integration with the so-called “omega arithmetics” is given at the end of this Section.

4.3.2.3.3 Signal scaling

The acceleration signals are either expressed in [g] or in [m/s²]. The first unit comes directly from the electrical characteristics of accelerometers, which are described by the so-called “sensitivity” - expressed in [mV/g]. The latter one, on the other hand, is the only unit, which is included in the worldwide International System of Units (SI), so in a sense is the only “legal” unit. The velocity signal is expressed in [mm/s]. Sometimes, the velocity-based values are further modified, like “**Velocity RMS**”, which means that each velocity value is post-processed.

Acceleration signal in [g] units is converted to [mm/s] according to following equation:

$$1g = 9.81 \frac{m}{s^2} = 9.81 \cdot 1000 \frac{mm}{s^2} = 9810 \frac{mm}{s^2},$$

which is expressed in MATLAB® simply as:

```
x_scaled = 9.81*1000*x; % convert [g] to [mm/s^2]
```

Note that the integration changes $\frac{1}{s^2}$ to $\frac{1}{s}$.

4.3.2.3.4 Resampling

Signal resampling is additionally used when Velocity Spectrum is calculated in the order domain. A complete, simplified resampling function in a form:

```
function [Xres,Nrot] = Resample_signal_simplified(x,PM,fs,Nsamp)
% INPUT
% x      - time signal <1xN>
% PM     - phase marker binary vector <1xN>
% fs     - sampling frequency <1x1>
% Nsamp  - new number of samples per rotation (optional) <1x1>

% OUTPUT
% Xres   - resampled signal <1xM>
% Nrot   - number of full rotations <1x1>
```

is given in Section 4.2.2.4.

4.3.2.3.5 Calculation of spectral amplitudes

When Velocity spectrum is calculated in frequency domain, the spectral amplitudes are for instance calculated in usual way:

```
Y = fft(x);
N = length(Y);
Y = abs(Y(1:N/2))/(N/2); % spectral, scaled amplitudes
```

For Velocity spectrum calculated in the order domain, the spectral amplitudes are calculated practically in the same way:

```
Y = fft(Xres_closed);
N = length(Y);
Y = abs(Y(1:N/2))/(N/2); % spectral, scaled amplitudes
```

4.3.2.3.6 Calculation of spectral resolution

When Velocity spectrum is calculated in frequency domain, the spectral resolution is calculated in usual way:

`resolution = 1/T or fs/length(x);`

Likewise, when Velocity spectrum is calculated in order domain, the spectral resolution is calculated in following way:

`resolution = 1/Nrot or Nsamp/length(Xres);`

Having the `resolution` and the `Y` variables, both, velocity spectrum in the frequency domain as well as velocity spectrum in the order domain could be plotted. Next Section illustrates examples of calculations of Velocity spectra.

4.3.2.4 Examples

4.3.2.4.1 List of signals

In this Section, three examples for velocity spectrum visualization are considered:

Simulated signal ***sig1***:

- sampling frequency 25 kHz
- length 10 seconds
- content sinusoidal component ($A = 7$, $f = 100$ Hz, phase shift = 0)
- constant speed **YES**

Simulated signal ***sig2***:

- sampling frequency 25 kHz
- length 10 seconds
- content sinusoidal component ($A = 7.13$, $f = 97.23$ Hz, phase shift = 0)
- constant speed **NO** (random fluctuations around 97.23 Hz “nominal speed”)

Real signal ***vibstand.mat***¹⁴:

- sampling frequency 25kHz,
- length 13 seconds
- content real
- constant speed **NO**

4.3.2.4.2 Single sinusoidal component

First example illustrates calculation of a velocity spectrum in frequency domain of a basic sinusoidal component with amplitude 7 and frequency 100 Hz. The resultant spectra are illustrated in Figure 4.44.

¹⁴ Available @ <http://home.agh.edu.pl/~ajab/>

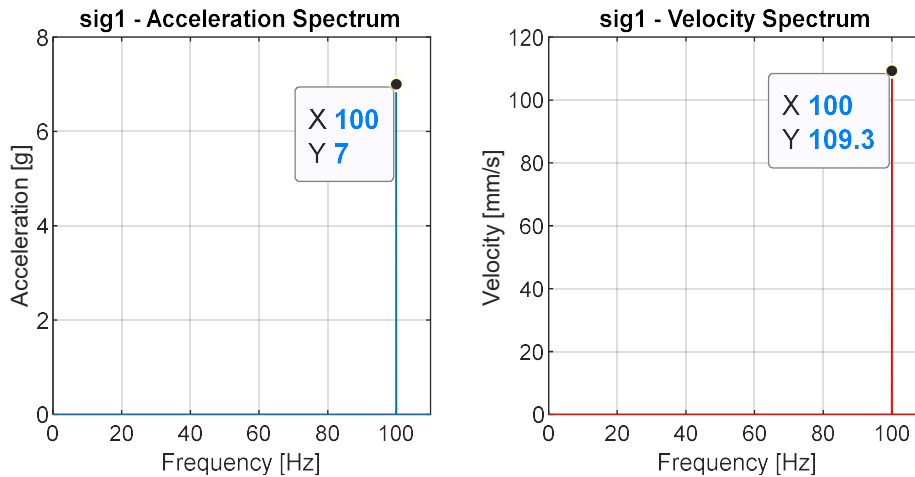


Figure 4.44 Acceleration and velocity spectrum of signal sig1

Note that for the sinusoidal component $sig1 = 7 \cdot \sin(2 \cdot \pi \cdot 100 \cdot t)$, the corresponding amplitude in velocity spectrum is calculates as:

$$7 \cdot \frac{1}{2 \cdot \pi \cdot 100} \cdot 9,81 \cdot 1000 = 109,2917$$

A complete source code for this test is given below:

```
% ----- Simulated signal "sig1" -----
% Data parameters
fs = 25e3;           % USER sampling frequency
T = 10;              % USER total signal length
dt = 1/fs;           % time resolution
t = dt:dt:T;         % time axis
N = length(t);       % number of points in the signal

% Generate and process data
sig1 = 7 * sin (2*pi*100*t);           % generate
sig1_HP_02 = Filter_signal_HP(sig1,fs,2); % filter HP (2Hz)
sig1_Int_02 = Calculate_Integrated(sig1_HP_02,fs); % integrate
sig1_Int_02_scaled = 9.81*1000*sig1_Int_02; % scale

% Calculate spectral amplitudes (acceleration and velocity)
Y_Acc = fft(sig1);           Y_Acc = abs(Y_Acc(1:N/2+1))/(N/2);
Y_Vel = fft(sig1_Int_02_scaled); Y_Vel = abs(Y_Vel(1:N/2+1))/(N/2);

% Display acceleration spectrum and velocity spectrum (zoom)
df = fs/N;
f = 0:df:fs/2;

figure;
subplot(121); plot(f,Y_Acc);           title('sig1 - Acceleration Spectrum');
ylabel('Acceleration [g]');           xlabel('Frequency [Hz]');
set(gca,'fontsize',8);               xlim([0 110]);

subplot(122); plot(f,Y_Vel,'r');       title('sig1 - Velocity Spectrum');
ylabel('Velocity [mm/s]');           xlabel('Frequency [Hz]');
set(gca,'fontsize',8);               xlim([0 110]); ylim([0 120]);
% ----- END -----
```

4.3.2.4.3 Variable speed component

The next example considers a sinusoidal component with non-integer amplitude and frequency values plus some frequency modulation (simulating speed fluctuation). The resultant spectra are illustrated in Figure 4.45.

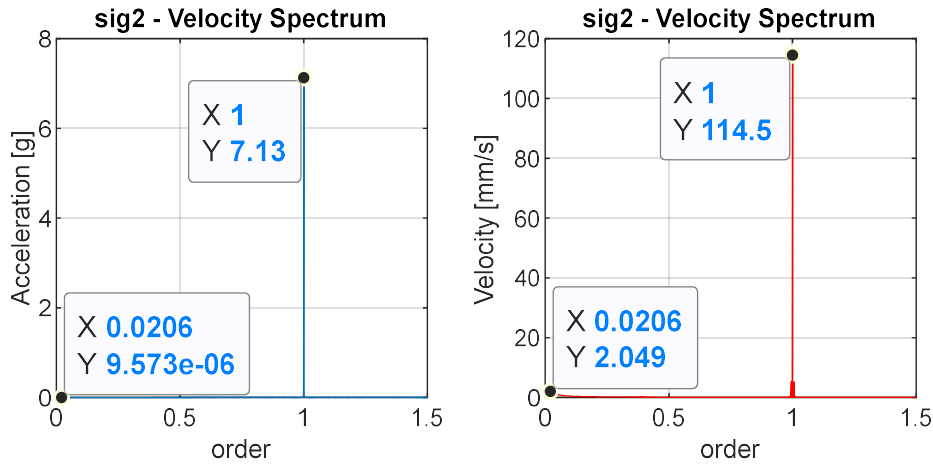


Figure 4.45 Acceleration and velocity spectrum of signal sig2

Note that for the sinusoidal component with a nominal speed, $sig2 = 7,13 \cdot \sin(2 \cdot \pi \cdot 97,23 \cdot t)$, the corresponding amplitude in velocity spectrum is calculates as:

$$7,13 \cdot \frac{1}{2 \cdot \pi \cdot 97,23} \cdot 9,81 \cdot 1000 = 114,4929$$

which is almost equal to the amplitude of the component in Figure 4.45. A complete source code for this test is given below:

```
% ----- Simulated signal "sig2" -----
% Data parameters
fs = 25e3;           % USER sampling frequency
T = 10;              % USER total signal length
dt = 1/fs;           % time resolution
t = dt:dt:T;         % time axis
N = length(t);       % number of points in the signal

FM_m = 9;             % USER speed fluctuation (i.e. FM) coefficient
profile = sin(2*pi*.5*t); % USER speed fluctuation (i.e. FM) profile

% Generate and process data
sig2 = 7.13 * sin(2*pi*97.23*t + 2*pi*FM_m*(cumsum(profile)*dt));

sig2_HP_02 = Filter_signal_HP(sig2,fs,2); % filter HP (2Hz)
sig2_Int_02 = Calculate_Integrated(sig2_HP_02,fs); % integrate
sig2_Int_02_scaled = 9.81*1000*sig2_Int_02; % scale

% Resample acceleration and velocity signals
% NOTE: The Nrot value is the same for Acc and Vel signals, so it is
% calculated only once
PM = Get_phase_markers(sig2); % see Section 4.4.4.2.2
[sig2_Res,Nrot] = Resample_signal_simplified(sig2,PM,fs); % Section 4.2.2.4
[sig2_Res_02] = Resample_signal_simplified(sig2_Int_02_scaled,PM,fs);
```

```

% Auxiliary variables
% NOTE: The Nres value is the same for Acc and Vel signals, so it is
%       calculated only once
Nres = length(sig2_Res); % number of points in resampled signal
Nsamp = Nres/Nrot; % number of samples per rotation (auto)

% Calculate spectral amplitudes (acceleration and velocity)
Y_res_Acc = fft(sig2_Res); Y_res_Acc=abs(Y_res_Acc(1:Nres/2+1))/(Nres/2);
Y_res_Vel = fft(sig2_Res_02); Y_res_Vel=abs(Y_res_Vel(1:Nres/2+1))/(Nres/2);

% Display acceleration spectrum and velocity spectrum (USER zoom)
d_ord = 1/Nrot; % corresponding to "df" in sig1
ord = 0:d_ord:Nsamp/2; % corresponding to "f" in sig1

figure;
subplot(121); plot(ord,Y_res_Acc); title('sig2 - Velocity Spectrum');
ylabel('Acceleration [g]'); xlabel('order');
set(gca,'fontsize',8); xlim([0 1.5]);

subplot(122); plot(ord,Y_res_Vel,'r'); title('sig2 - Velocity Spectrum');
ylabel('Velocity [mm/s]'); xlabel('order');
set(gca,'fontsize',8); xlim([0 1.5]); ylim([0 120]);
% ----- END -----

```

4.3.2.4.4 Real rotary machinery signal

The third example is a real signal from a Vibstand™ machinery. This example is used to illustrate the influence of the selection between two common HP cutoff frequencies used for velocity spectra calculations, namely:

- cutoff frequency 1 = 2Hz,
- cutoff frequency 2 = 10 Hz.

Both of these values come from the ISO vibration-based norm 20816 (former ISO 10816). The comparison is illustrated in Figure 4.46 (frequency domain) and in Figure 4.47 (order domain).

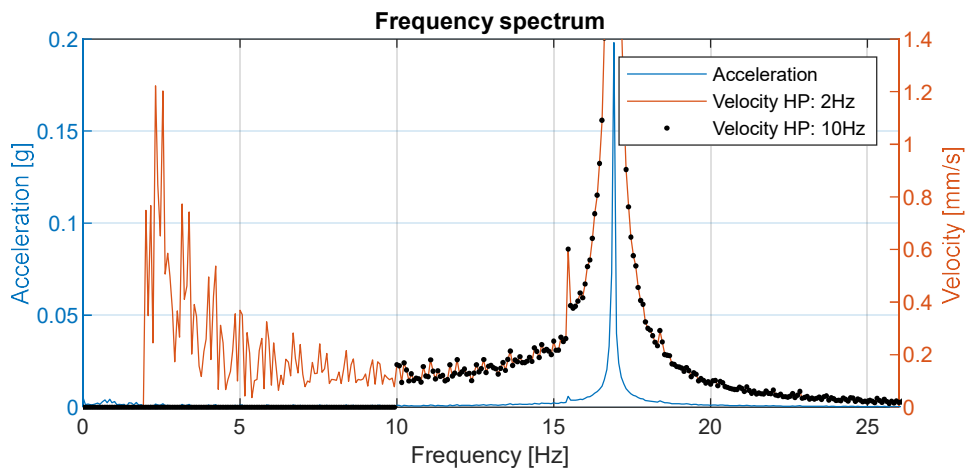


Figure 4.46 Comparison of the influence of HP cutoff frequency on velocity frequency spectrum

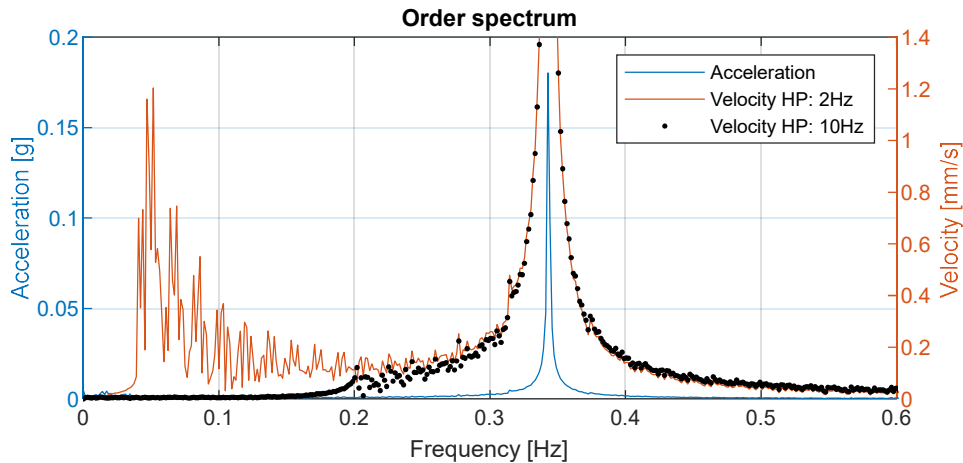


Figure 4.47 Comparison of the influence of HP cutoff frequency on velocity order spectrum

Clearly, for many instances, the HP cutoff frequency is simply not enough, because the velocity signal in relatively low frequencies quickly gets high values. Note that for frequency-domain filtering (FDF) used in this example, the frequency content of both velocity spectra is, of course, the same; however, in order domain it is slightly different due to accepted order of operations, i.e. integration followed by resampling. A complete source code for this test is given below:

```
% ----- Real signal "sig3" -----
% PART 1 - FREQUENCY DOMAIN
load('vibstand.mat');           % available @ http://home.agh.edu.pl/~ajab/
fs = 25e3;                      % 25 kHz sampling freq.
sig3 = avtb_2019_Machine_1_02_Motor_bearing(:,2); % vibration data

sig3_HP_02 = Filter_signal_HP(sig3,fs,2); % HP filter 2 Hz
sig3_HP_10 = Filter_signal_HP(sig3,fs,10); % HP filter 10 Hz

sig3_Int_02 = Calculate_Integrated(sig3_HP_02,fs); % integrate
sig3_Int_10 = Calculate_Integrated(sig3_HP_10,fs); % integrate

sig3_Int_02_scaled = 9.81*1000*sig3_Int_02; % scale first signal
sig3_Int_10_scaled = 9.81*1000*sig3_Int_10; % scale second signal

% Calculate spectral amplitudes
N = length(sig3);
Y = fft(sig3);
Y = abs(Y(1:N/2+1))/(N/2); % Acceleration - original signal

Y_02 = fft(sig3_Int_02_scaled); Y_02 = abs(Y_02(1:N/2+1))/(N/2); % Velocity
Y_10 = fft(sig3_Int_10_scaled); Y_10 = abs(Y_10(1:N/2+1))/(N/2); % Velocity

% Display two velocity spectra and reference acceleration spectrum (zoom)
df = fs/N;
f = 0:df:fs/2;

figure;
yyaxis left; plot(f,Y); ylabel('Acceleration [g]');
yyaxis right; plot(f,Y_02); hold on;
plot(f,Y_10,'k. '); ylabel('Velocity [mm/s]');
grid on;
xlim([0 26]);
ylim([0 1.4]);
```



```

xlabel('Frequency [Hz]');
title('Frequency spectrum');
legend('Acceleration', 'Velocity HP: 2Hz', 'Velocity HP: 10Hz');
set(gca, 'fontsize', 8);

% PART 2 - ORDER DOMAIN
PM = avtb_2019_Machine_1_02_Motor_bearing(:,1); % phase markers

% Resample acceleration and velocity signals
% NOTE: The Nrot value is the same for Acc and Vel signals, so it is
%       calculated only once
[sig3_Res, Nrot] = Resample_signal_simplified(sig3, PM, fs); % Section 4.2.2.4
[sig3_Res_02]    = Resample_signal_simplified(sig3_Int_02_scaled, PM, fs);
[sig3_Res_10]    = Resample_signal_simplified(sig3_Int_10_scaled, PM, fs);

% % Auxiliary variables
% NOTE: The Nres value is the same for Acc and Vel signals, so it is
%       calculated only once
Nres = length(sig3_Res); % number of points in resampled signal
Nsamp = Nres/Nrot;       % number of samples per rotation (auto)

% Calculate spectral amplitudes
Y_res    = fft(sig3_Res);    Y_res    = abs(Y_res    (1:Nres/2+1))/(Nres/2);
Y_res_02 = fft(sig3_Res_02); Y_res_02 = abs(Y_res_02(1:Nres/2+1))/(Nres/2);
Y_res_10 = fft(sig3_Res_10); Y_res_10 = abs(Y_res_10(1:Nres/2+1))/(Nres/2);

% Display two velocity spectra and reference acceleration spectrum (zoom)
d_ord = 1/Nrot; % corresponding to "df" in sig1
ord    = 0:d_ord:Nsamp/2; % corresponding to "f" in sig1

figure;
yyaxis left; plot(ord, Y_res); ylabel('Acceleration [g]');
yyaxis right; plot(ord, Y_res_02); hold on;
               plot(ord, Y_res_10, 'k'); ylabel('Velocity [mm/s]'); grid on;
               xlim([0 .6]); ylim([0 1.4]); xlabel('Frequency [Hz]');
               title('Order spectrum');
               legend('Acceleration', 'Velocity HP: 2Hz', 'Velocity HP: 10Hz');
               set(gca, 'fontsize', 8);

```

4.3.2.5 Influence of filtering domain

The signal processing paths illustrated so far in this Section use a custom algorithm for signal filtering in the frequency domain. However, sometimes a time-domain classical filtering needs to be used. Thus, it is important to investigate how the final velocity spectrum differs according to the selected HP filtering domain. Figure 4.48 illustrates a comparison between TDF and FDF for both HP filters, 2 Hz cutoff frequency and 10 Hz cutoff frequency.

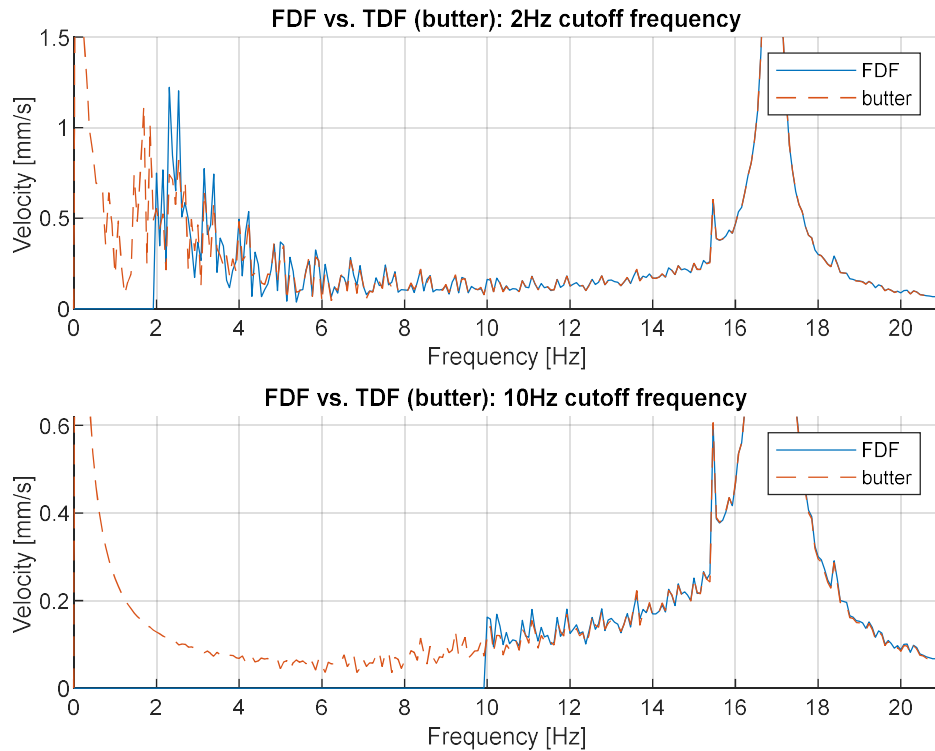


Figure 4.48 Comparison of time domain and frequency domain filtering

As illustrated in Figure 4.48, for both considered cutoff frequencies (2 Hz and 10 Hz), time-domain filters with relatively small order (here order 4), might easily generate high velocity values for relatively low-frequency components. A complete source code for this test is given below:

```
% ----- FDF vs. TDF -----
load('vibstand.mat');           % Available @ http://home.agh.edu.pl/~ajab/
fs = 25e3;                      % 25 kHz sampling freq.
sig3 = avtb_2019_Machine_1_02_Motor_bearing(:,2); % vibration data
N = length(sig3);               % number of points

% Filter the signal in the frequency domain
sig3_HP_02 = Filter_signal_HP(sig3,fs,2); % HP filter 2 Hz (FDF)
sig3_HP_10 = Filter_signal_HP(sig3,fs,10); % HP filter 10 Hz (FDF)

% Filter signal in time domain
[b,a] = butter(4,2/(fs/2),'high'); % filter order 4, cutoff freq. 2Hz
sig3_HP_02_butter = filter(b,a,sig3);

[b,a] = butter(4,10/(fs/2),'high'); % filter order 4, cutoff freq. 10Hz
sig3_HP_10_butter = filter(b,a,sig3);

% -----
% NOTE:
% When object-oriented filtering is used (for instance for 10 Hz cutoff
% frequency), like this:

% Fc = 10; % cutoff frequency 10 Hz
% fo = 4; % filter order: 4
% h_10 = fdesign.highpass('N,F3dB', fo, Fc, fs);
% Hd_10 = design(h_10, 'butter');
```

```

% sig3_HP_10_butter = filter(Hd_10,sig3);          % filter the signal (TD)

% Then, the resultant, filtered signal is slightly different than signal
% filtered using "butter" function. Worth mentioning, differences are
% clearly visible only for a few lowest frequencies of a full-resolution
% frequency spectrum. Practically, these differences are meaningless.
% -----

% Integrate
sig3_Int_02      = Calculate_Integrated(sig3_HP_02,fs); % FD-filt. signal
sig3_Int_10      = Calculate_Integrated(sig3_HP_10,fs); % FD-filt. signal
sig3_Int_02_butter = Calculate_Integrated(sig3_HP_02_butter,fs); % TD-filt.
sig3_Int_10_butter = Calculate_Integrated(sig3_HP_10_butter,fs); % TD-filt.

% Scale
sig3_Int_02_scaled      = 9.81*1000*sig3_Int_02;          % FD-filtered sig.
sig3_Int_10_scaled      = 9.81*1000*sig3_Int_10;          % FD-filtered sig.
sig3_Int_02_scaled_butter = 9.81*1000*sig3_Int_02_butter; % TD-filtered sig.
sig3_Int_10_scaled_butter = 9.81*1000*sig3_Int_10_butter; % TD-filtered sig.

% Calculate spectral amplitudes
Y_02      = fft(sig3_Int_02_scaled);          % FD-filtered signal
Y_10      = fft(sig3_Int_10_scaled);          % FD-filtered signal
Y_02      = abs(Y_02(1:N/2+1))/(N/2);
Y_10      = abs(Y_10(1:N/2+1))/(N/2);

Y_02_butter = fft(sig3_Int_02_scaled_butter);          % TD-filtered signal
Y_10_butter = fft(sig3_Int_10_scaled_butter);          % TD-filtered signal
Y_02_butter = abs(Y_02_butter(1:N/2+1))/(N/2);
Y_10_butter = abs(Y_10_butter(1:N/2+1))/(N/2);

% Generate frequency axis
df = fs/N;
f  = 0:df:fs/2;

% Display spectra
figure; subplot(211); hold on; grid on;
plot(f,Y_02);
plot(f,Y_02_butter,'--'); xlim([0 21]); ylim([0 1.5]);
title('FDF vs. TDF (butter): 2Hz cutoff frequency');
ylabel('Velocity [mm/s]'); xlabel('Frequency [Hz]');
legend('FDF','butter'); set(gca,'fontsize',8);

subplot(212); hold on; grid on;
plot(f,Y_10);
plot(f,Y_10_butter,'--'); xlim([0 21]); ylim([0 0.62]);
title('FDF vs. TDF (butter): 10Hz cutoff frequency');
ylabel('Velocity [mm/s]'); xlabel('Frequency [Hz]');
legend('FDF','butter'); set(gca,'fontsize',8);
% ----- END -----

```

4.3.2.6 Omega arithmetics

For quick velocity spectrum calculations, sometimes a so-called "Omega Arithmetics" is used, which comes directly from the analytical consideration of integration of a sinusoidal component, i.e.:

$$\int A \cdot \sin(2 \cdot \pi \cdot f) dt = -A \cdot \frac{1}{2 \cdot \pi \cdot f} \cdot \cos(2 \cdot \pi \cdot f)$$

Clearly, for spectral analysis of velocity signal this means that the frequency “candidates” remain the same and that each velocity amplitude is calculated from acceleration spectrum using $1/(2\pi f)$ coefficient. Figure 4.49 illustrates the comparison of a velocity spectrum calculated using FDF filtering and TD integration with the omega arithmetics algorithm:

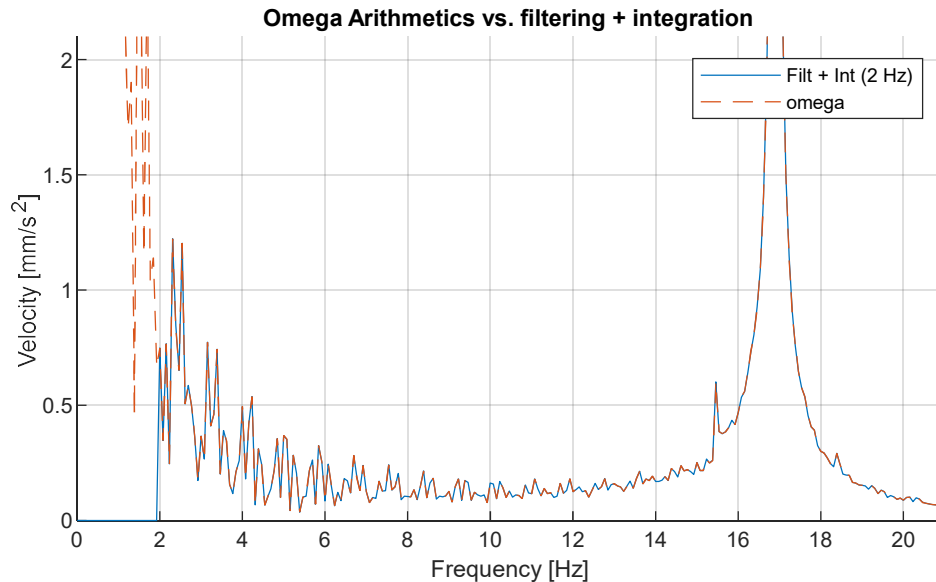


Figure 4.49 Omega Arithmetics vs. filtering + integration

Clearly, for some cases, omega arithmetics requires a suitable manual selection of the initial frequency range to be neglected; otherwise, spectral components might quickly reach very high values (as frequency f approach 0, the coefficient approaches ∞). However, the omega arithmetics has the advantage of being a reverse operation. A complete source code for this test is given below:

```
% ----- OMEGA ARITHMETICS DEMO -----
load('vibstand.mat'); % Available @ http://home.agh.edu.pl/~ajab/
fs = 25e3; % 25 kHz sampling freq.
sig3 = avtb_2019_Machine_1_02_Motor_bearing(:,2); % vibration data
N = length(sig3); % number of points

% Process data as already illustrated
sig3_HP_02 = Filter_signal_HP(sig3,fs,2); % filter HP (2Hz)
sig3_Int_02 = Calculate_Integrated(sig3_HP_02,fs); % integrate
sig3_Int_02_scaled = 9.81*1000*sig3_Int_02; % scale
Y_Vel = fft(sig3_Int_02_scaled);
Y_Vel = abs(Y_Vel(1:N/2+1))/(N/2); % Vel. spectral values
df = fs/N; % frequency resolution
f = 0:df:fs/2; % frequency axis

% Process data with "Omega Arithmetics"
Y_Acc = fft(sig3); % Regular "acceleration" processing
Y_Acc = abs(Y_Acc(1:N/2+1))/(N/2); % Acceleration spectral values

f_omega = df:df:fs/2; % "Omega" frequency axis
Y_omega = Y_Acc(2:end)'./(2*pi*f_omega); % Omega Arithmetics
Y_omega_scaled = 9.81*1000*Y_omega; % Scale Acc. to Vel.

% Compare velocity spectra (zoom)
figure; hold on; grid on;
```

```

plot(f,Y_Vel);
plot(f_omega,Y_omega_scaled,'--');

xlim([0 21]);          % USER zoom
ylim([0 2.1]);         % USER zoom

title('Omega Arithmetics vs. filtering + integration');
ylabel('Velocity [mm/s]'); xlabel('Frequency [Hz]')
legend('Filt + Int (2 Hz)', 'omega'); set(gca,'fontsize',8);
% ----- END -----

```

Following Section illustrates further concept of “displacement spectrum”.

4.3.2.7 Displacements spectrum

4.3.2.7.1 General concept

Section 4.3.2.7.1 - 4.3.2.7.3 described how velocity frequency spectrum and velocity spectrum in order domain could be generated from a raw acceleration signal. The basic physical phenomenon used in both cases is the fact that velocity is the “integral” of the acceleration. Because analogously displacement is the “integral” of the Velocity, symbolically written as:

$$Displacement [mm] = \int Velocity \frac{mm}{s} = \int Acceleration [g]$$

it seems possible to calculate the displacement performing another integration on the velocity signal.

Note that the conversion from acceleration in [g] to velocity in [mm/s] included additional scaling due to conversions of units; this time it is not necessary. For conciseness, this Section considers two cases:

Simulated signal **sig1**:

- sampling frequency 25 kHz
- length 10 seconds
- content sinusoidal component (A = 7, f = 100 Hz, phase shift = 0)
- constant speed **YES**
- HP cutoff frequency 2 Hz

Real signal **vibstand.mat**¹⁵:

- sampling frequency 25kHz,
- length 13 seconds
- content real
- constant speed **NO**
- HP cutoff frequency 10 Hz

Note that in case of displacement spectrum, it is recommended to verify the time characteristics of the signal, as well.

¹⁵ Available @ <http://home.agh.edu.pl/~ajab/>

4.3.2.7.2 Single sinusoidal component

For the first example with **sig1** (used previously for velocity spectrum, the amplitude of the single spectral component $7 \cdot \sin(2 \cdot \pi \cdot f)$ is calculated as follows:

```
Acc = 7;
Vel = 7*1/(2*pi*100)*9810; % Vel == 109.2917
Dis = Vel * 1/(2*pi*100); % Dis == 0.1739
```

Figure 4.50 illustrates time waveforms and spectral representations of Acceleration, Velocity, and Displacement signals.

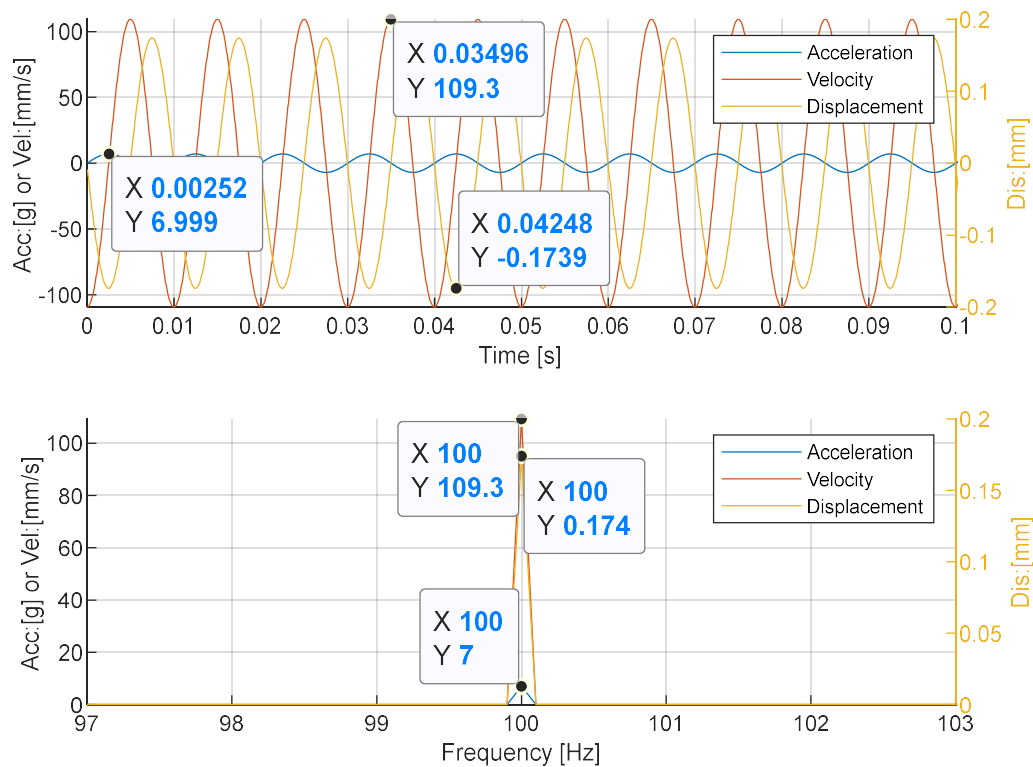


Figure 4.50 Displacement spectrum of *sig1*

Figure 4.50 is generated using following code:

```
% ----- Simulated signal "sig1" -----
% Data parameters
fs = 25e3; % USER sampling frequency
T = 10; % USER signal length
dt = 1/fs; % time resolution
t = dt:dt:T; % time axis
N = length(t); % number of points in the signal

% Calculate Velocity signal
sig1 = 7 * sin(2*pi*100*t); % generate
sig1_HP = Filter_signal_HP(sig1,fs,2); % filter HP (2Hz)
sig1_Int = Calculate_Integrated(sig1_HP,fs); % integrate
sig1_Int_scaled = 9.81*1000*sig1_Int; % scale
```

```

% Calculate Displacement signal
sig1_Int_Int = Calculate_Integrated(sig1_Int_scaled,fs); % integrate

% Calculate spectral amplitudes (acceleration, velocity, and displacement)
Y_Acc = fft(sig1); Y_Acc = abs(Y_Acc(1:N/2+1))/(N/2);
Y_Vel = fft(sig1_Int_scaled); Y_Vel = abs(Y_Vel(1:N/2+1))/(N/2);
Y_Dis = fft(sig1_Int_Int); Y_Dis = abs(Y_Dis(1:N/2+1))/(N/2);

% Display time waveforms
subplot(211);
yyaxis left; hold on;
plot(t,sig1, '-', 'color',[0 ,.447,.741]); % Acc
plot(t,sig1_Int_scaled, '-', 'color',[.85,.325,.098]); % Vel
xlabel('Time [s]');
ylabel('Acc:[g] or Vel:[mm/s]');
set(gca,'YColor',[.15,.15,.15]);

yyaxis right; plot(t,sig1_Int_Int, 'color',[.929,.694,.125]); % Dis
ylabel('Dis:[mm]');
set(gca,'YColor',[.929,.694,.125]);
legend('Acceleration','Velocity','Displacement');
set(gca,'fontsize',8);
xlim([0 .1]); grid on;

% Spectral resolution and axis
df = fs/N;
f = 0:df:fs/2;

% Display acceleration spectrum and velocity spectrum (USER zoom)
subplot(212);
yyaxis left; hold on;
plot(f,Y_Acc, '-', 'color',[0 ,.447,.741]); % Acceleration
plot(f,Y_Vel, '-', 'color',[.85,.325,.098]); % Velocity
ylabel('Acc:[g] or Vel:[mm/s]');
xlabel('Frequency [Hz]');
set(gca,'YColor',[.15,.15,.15]);

yyaxis right; plot(f,Y_Dis, 'color',[.929,.694,.125]); % Displacement
ylabel('Dis:[mm]');
legend('Acceleration','Velocity','Displacement');
set(gca,'fontsize',8);
set(gca,'YColor',[.929,.694,.125]);
xlim([97 103]);
grid on;

% ----- END -----

```

4.3.2.7.3 Real rotary machinery signal

Section 4.3.2.4.4 demonstrates how to calculate a velocity spectrum from a real vibration signal collected from a Vibstand™ test rig. This Section illustrates, how this signal could be integrated further to calculate the displacement spectrum. The velocity data is also displayed for reference. Figure 4.51 illustrates all three time waveforms. Top graph illustrates original acceleration signal with a clear Imbalance component (roughly 16 Hz). The bottom graphs show velocity and displacement signals.

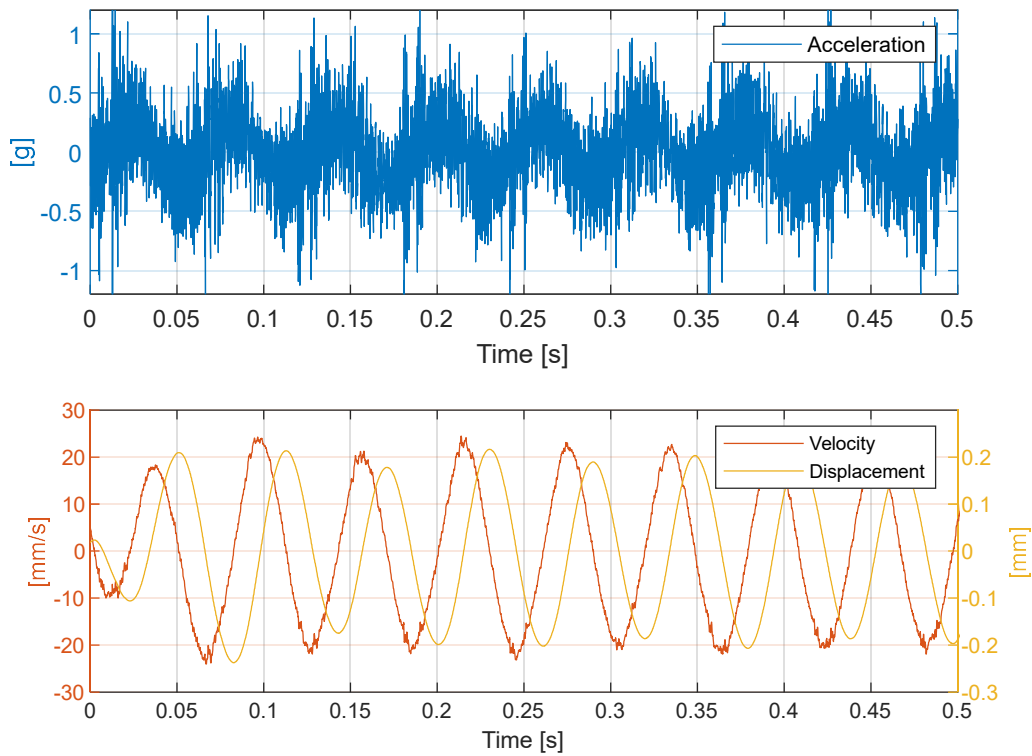


Figure 4.51 Time waveforms of *sig3*: Acceleration (top), Velocity (bottom), and Displacement (bottom)

As observed in Figure 4.51, the Velocity and Displacement waveforms present the Imbalance-generated component more clearly than the acceleration signal, which is a fundamental phenomenon used in vibration-based diagnostics. Also, note the phase shift resulting from fundamental trigonometric integral properties:

$$\text{Acceleration} \rightarrow \text{Velocity:} \quad \int \sin = -\cos$$

$$\text{Velocity} \rightarrow \text{Displacement:} \quad \int -\cos = -\int \cos = -\sin$$

Moreover, in practical implementations, the initial part of velocity and displacement signals are deleted due to initial signal distortion, as illustrated in Figure 4.51.

Figure 4.51 is generated with following code:

```
% ----- Real signal "sig3" -----
load('vibstand.mat');           % Available @ http://home.agh.edu.pl/~ajab/
fs = 25e3;                      % sampling frequency
sig3 = avtb_2019_Machine_1_02_Motor_bearing(:,2); % vibration data
N = length(sig3);               % No. of point
dt = 1/fs;                      % time resolution
T = N*dt;                      % signal total time
t = dt:dt:T;                   % time axis

% Calculate Velocity signal
sig3_HP = Filter_signal_HP(sig3,fs,10); % filter HP (10Hz)
sig3_Int = Calculate_Integrated(sig3_HP,fs); % integrate
sig3_Int_scaled = 9.81*1000*sig3_Int; % scale
```



```

% Calculate Displacement signal
sig3_Int_Int = Calculate_Integrated(sig3_Int_scaled,fs); % integrate

% Display time waveforms
figure;
subplot(211); plot(t,sig3); % Acceleration - original waveform
xlim([0 .5]); ylim([-1.2 1.2]);
ylabel('[g]'); xlabel('Time [s]'); legend('Acceleration');
set(gca,'YColor',[0,.447,.741]); grid on;

subplot(212);
yyaxis left; plot(t,sig3_Int_scaled, '-', 'color',[.85,.325,.098]); % Vel
ylabel('[mm/s]'); xlabel('Time [s]');
set(gca,'YColor',[.85,.325,.098]);
ylim([-30 30]); grid on

yyaxis right; plot(t,sig3_Int_Int, 'color',[.929,.694,.125]); % Dis
ylabel('[mm]'); legend('Velocity','Displacement');
set(gca,'fontSize',8); set(gca,'YColor',[.929,.694,.125]);
xlim([0 .5]); ylim([-0.3 .3]);

```

Figure 4.52 illustrates all three full-resolution spectra. Top graph illustrates a spectrum from original acceleration signal. The central graph shows spectra from velocity signal, and finally the bottom graph show spectrum generated from the displacement signal.

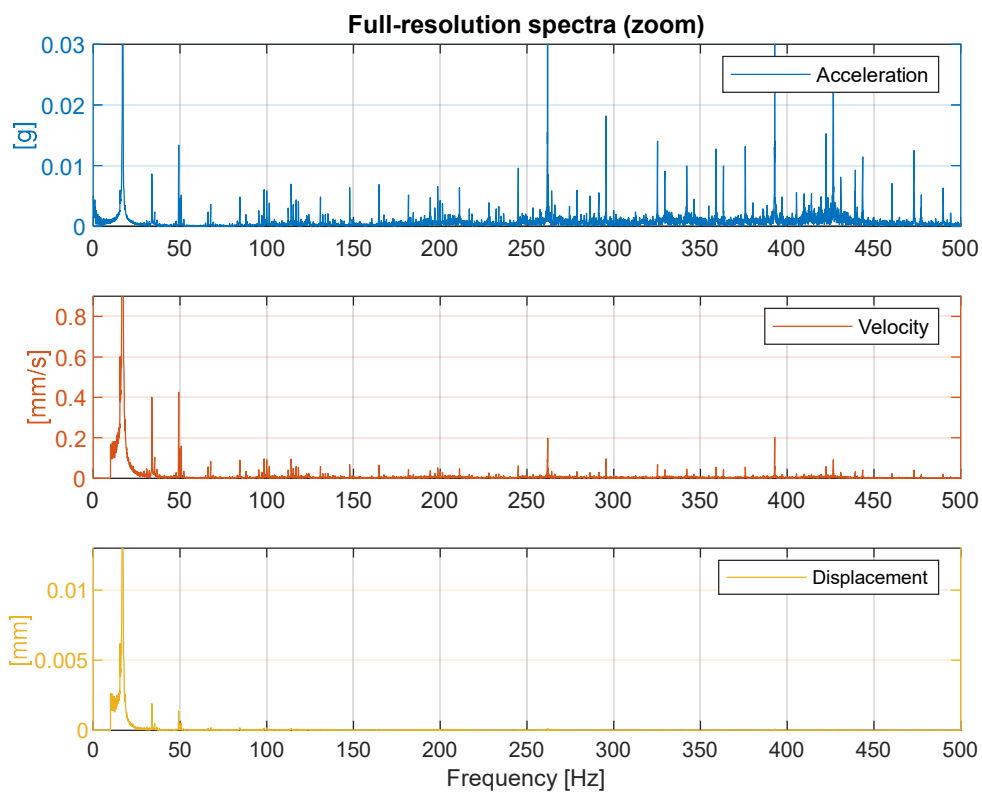


Figure 4.52 Spectra of *sig3*: Acceleration (top), Velocity (bottom), and Displacement (bottom)

As observed in Figure 4.52, analogously to the time waveform, velocity and displacement spectra present the Imbalance-generated component more clearly than the acceleration spectrum. Figure 4.52 is generated with following code:

```
% #1: FULL RESOLUTION spectra
% Calculate spectral amplitudes (acceleration, velocity, and displacement)
Y_Acc = fft(sig3);          Y_Acc = abs(Y_Acc(1:N/2+1))/(N/2);
Y_Vel = fft(sig3_Int_scaled); Y_Vel = abs(Y_Vel(1:N/2+1))/(N/2);
Y_Dis = fft(sig3_Int_Int);  Y_Dis = abs(Y_Dis(1:N/2+1))/(N/2);

% Calculate frequency scale
df = fs/N;
f = 0:df:fs/2;

% Display spectra (zoom)
figure;
subplot(311); plot(f,Y_Acc); % Acceleration - original waveform
xlim([0 500]); ylim([0 .03]);
title('Full-resolution spectra (zoom)');
ylabel(' [g] '); legend('Acceleration');
set(gca,'YColor',[0,.447,.741]); grid on;

subplot(312); plot(f,Y_Vel,'-','color',[.85,.325,.098]); % Velocity
xlim([0 500]); ylim([0 .9]);
ylabel(' [mm/s] '); legend('Velocity');
set(gca,'YColor',[.85,.325,.098]); grid on;

subplot(313); plot(f,Y_Dis,'color',[.929,.694,.125]); % Displacement
xlim([0 500]); ylim([0 .013]);
ylabel(' [mm] '); legend('Displacement');
set(gca,'YColor',[.929,.694,.125]);
xlabel('Frequency [Hz]'); set(gca,'fontsize',8); grid on;
```

A complementary comparison of the acceleration, velocity, and displacement spectra could be performed using *pwech* spectra, which typically are potted in *dB* scale, and illustrated in Figure 4.53.

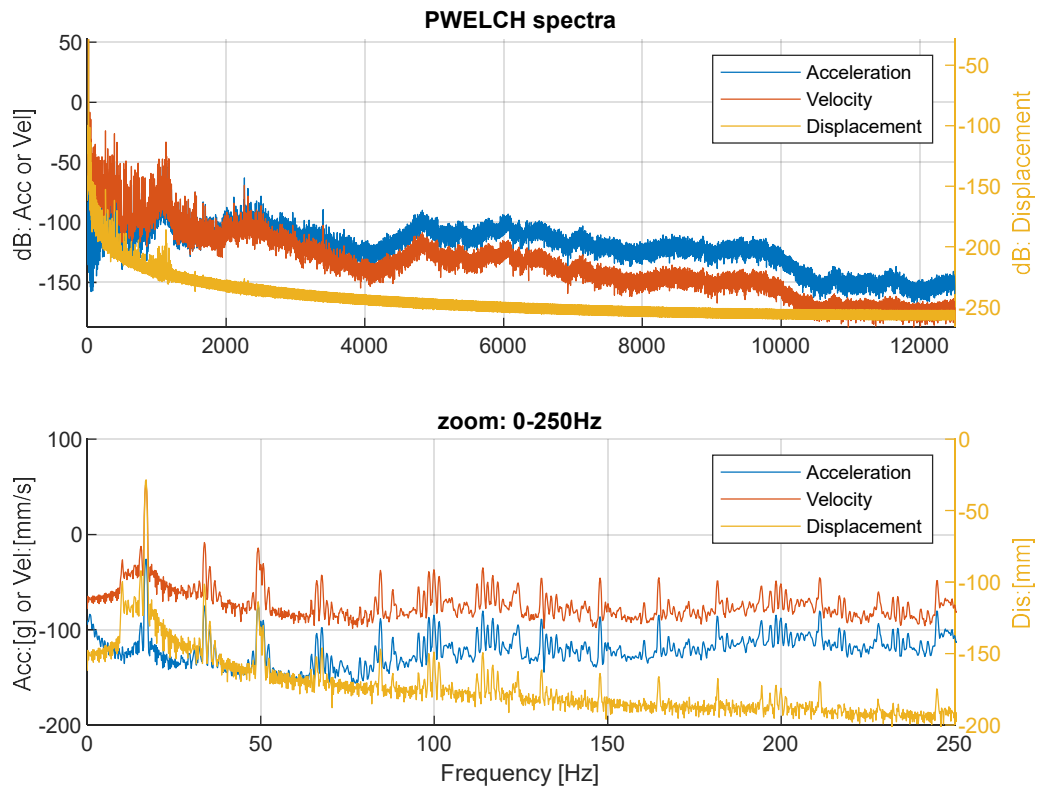


Figure 4.53 Spectra of *sig3* in dB scale: Acceleration, Velocity, and Displacement

Clearly, the values of displacement spectral components fall very quickly comparing to acceleration and velocity spectra.

For this reason, in practical analysis, velocity spectrum is typically used to track frequency (or their order counterparts) components in the range 2-150 Hz, while the displacement spectrum generated by double integration of the original acceleration signal is used just in special situations.

The code for dB-spectra generation, presented in Figure 4.53 is as follows:

```
% #2: PWELCH (dB)
% Calculate spectral amplitudes (acceleration, velocity, and displacement)
[pxx_Acc,f_pxx] = pwelch(sig3, [], [], [], fs);
[pxx_Vel      ] = pwelch(sig3_Int_scaled, [], [], [], fs);
[pxx_Dis      ] = pwelch(sig3_Int_Int,   [], [], [], fs);

% Display spectra
figure;
subplot(211);
yyaxis left;hold on;
    plot(f_pxx,20*log10(pxx_Acc), '-','color',[0 ,.447,.741]);%Acc
    plot(f_pxx,20*log10(pxx_Vel), '-','color',[.85,.325,.098]);%Vel
    title('PWELCH spectra'); ylabel('dB: Acc or Vel');
    set(gca,'YColor',[.15,.15,.15]);

yyaxis right;plot(f_pxx,20*log10(pxx_Dis), 'color',[.929,.694,.125]); %Dis
    ylabel('dB: Displacement');
    legend('Acceleration','Velocity','Displacement');
    set(gca,'fontsize',8); set(gca,'YColor',[.929,.694,.125]);
```

```

axis tight; grid on;

subplot(212);
yyaxis left; hold on;
    plot(f_pxx, 20*log10(pxx_Acc), '-', 'color', [0 , .447, .741]); %Acc
    plot(f_pxx, 20*log10(pxx_Vel), '-', 'color', [.85, .325, .098]); %Vel
    ylabel('Acc:[g] or Vel:[mm/s]'); xlabel('Frequency [Hz]');
    set(gca, 'YColor', [.15, .15, .15]);

yyaxis right; plot(f_pxx, 20*log10(pxx_Dis), 'color', [.929, .694, .125]); %Dis
    ylabel('Dis:[mm]'); title('zoom: 0-250Hz');
    legend('Acceleration', 'Velocity', 'Displacement');
    set(gca, 'fontsize', 8); set(gca, 'YColor', [.929, .694, .125]);
    xlim([0 250]); ylim([-200 0]); grid on;

```

4.3.3 Time Synchronous averaging (TSA)

4.3.3.1 Overview

From CMS algorithm development point-of-view, this Section is a continuation of the basic information presented in Section 4.2.2.

Time Synchronous Averaging (TSA) is a powerful vibration signal analysis tool, which enables extraction of periodic, so called phase-locked signal components from a noisy vibration signal or from other phase-locked components (with other periods). This technique is particularly useful for extraction of gearbox-generated periodic components, like gear meshing components (GMF) [8], but it also could be used for detection of REB faults [9]. In general, TSA averages a vibration signal with respect to some referential signal. Probably most typically, the referential signal is a one-per revolution “pulse”, which is called a “phase marker”. This topic is covered in details in Section 4.2.2. Therefore, although the name of the technique includes “time”-based averaging, it might be somehow confusing, because typically the averaging within TSA is rather rotational (angle-based).

The current Section first includes analysis of **resampled signal**, as a **crucial TSA** step. Next, selected aspects of resampled signal spectral analysis are presented, as an extension to Sections 4.2.2, where basic theory on signal resampling is covered. Finally, order spectrum of TSA signal is compared with both, frequency and order spectrum. Since practical TSA analysis frequently includes averaging the same vibration signal (with a single PM referential speed data) with respect to different periods (i.e. shafts), this scenario is considered throughout this Section. This is what differs the presented codes from the MATLAB® `tsa` function briefly reviewed at the beginning of this Section. Additional consideration on TSA, including analysis and comparison of different numerical techniques are found in [10].

4.3.3.2 Time-domain and angle-domain waveforms

4.3.3.2.1 MATLAB® “tsa” function

The general concept of calculation of the time-synchronous signal via MATLAB® `tsa` function is essentially the same as calculation using user’s code presented in this Section, and includes following steps:

- 1) signal trimming,
- 2) signal resampling (2 x signal interpolation),
- 3) signal reshaping
- 4) signal averaging (along rotations).